

一面 60min

自我介绍 + 竞赛经历

项目简单拷打

这一部分比较简单，就是问了问解决了什么问题，记忆怎么处理的，顺带提了一些Agent的八股，有没有用现成 Agent 框架。

Go 八股的简单拷打

GMP, goroutine

(有点不理解，他们作为一个python岗为什么要拷打我go的八股)

开源相关经历的拷打

这里说的是参考了他们的设计，所以顺带修复了一些bug，提了pr
不过k8s那一部分的开源经历说的不行，回去补一下。

主要就是以下几个问题。

1. 是你项目里用到这些组件后发现 bug 才提 PR，还是主动找 issue?
2. 为什么会去看 JuiceFS?
3. 为什么会关注 Kubernetes?
4. 你做开源贡献的动机是什么?
5. 是为了 GSoC，还是为了简历，还是为了解决真实问题?

算法&数据结构部分

1. 左右括号匹配 (使用栈)
2. top10 (这个回答的有问题，只想到了用快速排序，没有想到用小根堆)

场景设计题

并发限流锁, redis并发限流

这一部分做的非常差劲，不过好在有一点点对的地方，外加大二年纪有限这个免死金牌，勉强过关

聊天环节

这个没啥好说的，就是你怎么看自己学校课程和外部环境的差距？Python 写过吗？你后面有考研规划吗？你现在有没有其他 offer？怎么review代码？怎么控制代码生成的边界？技术选型是怎么做的？为什么选择 pgvector，而不是 Milvus？

反问环节

感谢joye哥的面经，这部分还是很稳的。

- 公司有没有内部的 tool/function 库？
- Agent 项目偏 ToC 还是 ToB？
- 目前有没有已落地场景？
- 用户画像是什么？
- 实习生进来三个月内主要解决什么问题？
- 面试官觉得你表现好的地方和不足是什么？

二面50min

依旧自我介绍....

项目拷打

角色属性、血量、攻击属性怎么保存？记忆问题？RAG 库相关的内容（这部分为了防止拷打我八股，我直接把我所有会的八股一股脑融进了我的项目介绍里，如果对自己项目没有信心的不要这么做）有没有用现成 Agent 框架？Agent Runtime 与工具编排相关内容也拷打了不少，这部分占了很长一段时间，几乎是这次二面的核心内容。大概问题如下：

1. Agent Runtime 与工具编排

1. 你提到的 Agent Runtime 和工具编排具体做了什么？
2. Runtime 里有哪些模块？
3. 工具有哪些？
4. 骰子工具怎么设计？
5. 检查规则工具怎么设计？
6. 攻击/扣血/属性增长怎么做？
7. 什么是高层工具？
8. 为什么要把多个工具调用合并成一个高层工具？

9. 工具怎么注册到提示词里？
10. RAG 检索是否也包含在 Runtime 里？
11. Runtime 里是否包含评估和监控指标？
12. QPS、P95、P99 是否在 Runtime 体系里？

2. 架构图与完整链路

1. 你方便投屏画一下架构吗？
2. 这个 Runtime 的整体结构是什么？
3. RAG、Model、HTTP、Metrics 分别负责什么？
4. 前端请求进来后，完整链路怎么走？
5. HTTP 层做什么？
6. 鉴权之后进入哪里？
7. 为什么要创建 message job？
8. 为什么要有 outbox？
9. PG 是什么意思？
10. message 从 HTTP 进来后，整理哪些内容？
11. message 里除了文本，还有哪些字段？
12. 为什么要有 user_id、session_id、timestamp？
13. 有没有全局 ID？
14. 一个全局 session 下面是不是绑定多个 user？

3. Message Job / Outbox 机制

1. message job 是什么？
2. outbox 是什么？
3. message job 和 outbox 分别在 PG 里吗？
4. message job 有哪些状态？
5. pending、processing、fail、complete 分别是什么意思？
6. fail 是否区分可重试和不可重试？
7. outbox 怎么防止消息发送过程中宕机导致消息丢失？
8. outbox 是不是类似本地消息表/事务表？
9. outbox 如何扫描 pending 或 retryable failed 消息？
10. 重试逻辑放在 outbox 还是 RabbitMQ？

4. RabbitMQ 追问

1. 为什么请求不直接到模型层，而是要发到 RabbitMQ？
2. 引入 RabbitMQ 解决了什么问题？
3. 如果直接调模型，会造成什么阻塞？
4. Go routine 能不能解决这个问题？
5. 为什么只有 goroutine 还不够？
6. 多 user 同 session 下，如何避免互相阻塞？
7. RabbitMQ 如何提升异步处理能力？
8. RabbitMQ 的交互链路是怎么样的？
9. RabbitMQ publish 后，model 怎么消费？
10. model 处理完成后如何更新 message job？
11. 如果 model 处理失败，状态怎么变？
12. 为什么不在 MQ 层做复杂重试？
13. MQ 上游和模型下游是不是不同服务？
14. 模型服务怎么访问 message job/outbox 表？

5. 模型服务与状态更新

1. 模型层怎么更新 message job 表？
2. 是根据模型结构化输出里的 user_id/message_id 更新吗？
3. 外围业务代码是调接口，还是直接写库？
4. 模型服务是不是也连数据库？
5. 这样服务边界是否耦合？
6. 如果模型输出格式不符合预期怎么办？
7. parser 解析失败后怎么办？
8. 会不会再次调用大模型让它重新输出？
9. 提示词是否完全可信？
10. 如何做边界控制和失败冗余？

6. RAG 追问

1. RAG 部分怎么做？
2. 数据怎么获取？
3. 数据怎么清洗？
4. chunk 怎么切？
5. 规则库和设定库的 chunk 策略是否不同？
6. 为什么规则库按段落切？
7. 为什么设定库按章节切？
8. embedding 怎么做？

9. 向量索引怎么存?
10. 为什么用 pgvector?
11. 为什么不用 Milvus?
12. 数据量不大时, 怎么避免系统复杂度过高?
13. RAG 怎么回测?
14. 用了哪些指标?
15. Recall@N 是什么?
16. MRR 是什么?
17. 混合检索是怎么做的?
18. lexical/BM25 和 embedding 怎么结合?
19. 规则库和设定库的 MRR/Recall 表现如何?
20. 为什么设定库 MRR 能达到 1.0?
21. 在线测试怎么验证 RAG 能检索到正确设定?

7. 上下文管理 / 记忆机制

1. 上下文管理还有什么设计?
2. 滑动窗口怎么做?
3. compact 什么时候触发?
4. 超过 20 轮会不会压缩?
5. token/字数过长会不会压缩?
6. 压缩后的内容存在哪里?
7. 滑动窗口往后移时, 前面的消息是不是直接丢?
8. 如果上一个窗口丢了, 下一个窗口怎么感知前面的内容?
9. PG 是否保存完整历史?
10. PG 是否是唯一可信数据源?
11. 压缩的是内存里的 message list 吗?
12. LLM summary 如何保留前面轮次的关键信息?
13. 有没有短期记忆和长期记忆?
14. 用户状态算长期记忆吗?
15. 血量、攻击、属性这些长期状态存在哪里?
16. 项目里有没有类似 OpenAI/Hamers 那种 learning loop 的记忆?

8. Metrics / 监控 / 压测

1. Metrics 是旁路的吗，还是实时打点？
2. 打点逻辑在哪里？
3. 是函数开始打点，还是函数结束打点？
4. 成功/失败时是否打点？
5. 业务代码是否感知打点？
6. QPS 是否有统计？
7. QPS 是 Runtime 里实时统计，还是单独压测？
8. 为什么 QPS 要通过 HTTP 压测，而不是直接调函数？
9. P95、P99 怎么看？
10. 每一步 ReAct 是否记录工具调用、当前 step、max step？
11. 是否能看到消息运行到了哪一步？
12. 当前服务是否健康运行怎么判断？

万幸，基本都接住了....呼....

如果你的项目有信心非常夯，而且也经得起拷打，那么可以学我这样把所有的八股全部塞进项目里面去讲，自己掌握主导权，一定别怂，牛逼的地方就得吹牛，不然你做出什么来干什么？

然后开始了喜闻乐见的聊天环节

- 为什么你是信息安全专业，却不走网安方向？（因为转专业，第一年的成绩有限，没法转到计算机科学与技术，所以被迫退而求其次）
- 转专业，而且才大二还能做到这个水平？怎么做到的？（把我从最早开始接触编程到后来准备投入这个方向，以后怎么打算现场搓了个陈情表。面试官评价为满分回答）
- 为什么更想做后端/Agent？（工程化才是我追求的东西，我对这个系统维稳和提供服务有着极大的热情）
- 你是从哪里学习这些工程知识的？（黑皮书，了解一些新东西后继续去深度学习，不浅尝辄止）
- 你是否一直在做题，还是更多靠项目驱动？（项目驱动，不喜欢做题，就像前面说的，工程化才是追求，不落地永远不知道哪里会有问题）
- 你对自己当前能力怎么看？（同校同龄可能算是顶尖，但是必须把自己放到大环境中校准，这也是为什么我一直想要加入真实的业务环境，现在来看大约在中流偏上一点点。然后面试官说，不用太谦虚，你是我面试的里面非常顶尖的一批了，不出意外的话二面通过了，可以等三面通知了）（听到这句话我直接心里：！？卧槽？！）

总结：

项目真实性非常重要！

这两轮中厂级别的面试让我意识到这个项目的真实性是非常重要的，虽然大家都在说，“啊大不了抄个项目然后背就完了”。我在这里持绝对的反对态度，我严重抵制这种行为，虽然只是我个人而言。如果这个项目是你做的，而且有非常多可以讲的地方，对于这种中厂而言你甚至可以把原本八股的时间全部挤掉，一直讲你的项目，掌握主动权。只有你主动，才能把这个面试的评分尽可能拉高。

Agent的理解

很明显，目前agent从无到有只有两三年的时间，一切都在变化，大家都没有一套完整的理解和合适的架构设计，所以跟上主流是很重要的，openclaw，cc，hermes这些东西还是要知道怎么一回事的。哪怕你只是在面试中提一嘴对比一下，说不定也是能加分的。

agent的大脑是LLM，这个本质上非常像一个黑盒，不过我们要做的就是让黑盒变得可控，所以不能完全相信LLM的输出，对于它的输出一定要有一个检验和重试，否则那就是拿着项目当玩具，能跑最好跑不起来无所谓，这种态度我认为是不可取的。

工程链路一定要绝对的清晰和完全的掌握，别自己的项目都说不出来，那不是纯傻x嘛？

自己写的项目明明是一个加分点，如果你答不上来主要的链路就会变成扣分点，面试官会认为你这个项目就是vibe coding出来的。这位先生，你也不希望你花时间做的工作变成扣分项吧？

自己别当傻x，也别把别人当傻x。

一定要有逻辑

比如讲链路的时候就说第一步，第二步，第三步。

讲技术选型就先说结论，再说背景，然后阐述对比效果，最后解释选型内容，如果可以的话可以说一嘴自己选型取舍带来的潜在问题。

总而言之，一定要让自己有逻辑，一边说一边思考没有关系，但是一定要有逻辑，表现出工程师的潜质。

最后小结：尽人事，看天命。我觉得我做的已经很不错了，但是他们公司用的是python，外加我是七月到岗可能有点晚，所以二面结束之后hr来找我收集信息之后一直没有三面也情有可原，不过不是我的问题。